

7. Integrátory v molekulární dynamice

- MD modeluje realitně časový poměr Newtonových rovnic
- stísební hodnoty veličin určující časový stěračník
- deterministická metoda
- vhodné pro rovnovážné i nerovnovážné simulace
- vhodné pro nespojitě potenciály

Verletův bezrychlostní algoritmus

→ platí:

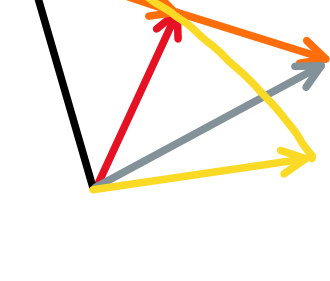
$$r_i(t+\Delta t) = r_i(t) + v_i(t)\Delta t + \frac{1}{2} a_i(t) \Delta t^2$$

$$r_i(t-\Delta t) = r_i(t) - v_i(t)\Delta t + \frac{1}{2} a_i(t) \Delta t^2$$

$$\Rightarrow r_i(t+\Delta t) = 2r_i(t) - r_i(t-\Delta t) + \underbrace{a_i(t) \Delta t^2}_{\text{zisk ze sil stojí výpočetní čas}}$$



$$r_i(t+\Delta t) = \underbrace{r_i(t)}_{0.\text{řád}} + \underbrace{[r_i(t) - r_i(t-\Delta t)]}_{1.\text{řád}} + \underbrace{a_i(t) \Delta t^2}_{2.\text{řád}}$$



→ simulace je potřeba inicializovat

$$r_i(-\Delta t) = \underbrace{r_i(0)}_{\text{poč. podm.}} - \underbrace{v_i(0)\Delta t}_{\text{výpočet } v_0} + \frac{1}{2} \underbrace{a_i(0) \Delta t^2}_{\text{výpočet } v_0}$$

→ pro výpočet energie používáme $v_i(t) = \frac{r_i(t+\Delta t) - r_i(t-\Delta t)}{2\Delta t}$

→ symetrie vůči $\Delta t \rightarrow -\Delta t \Rightarrow$ zachování energie

$$r_i(t-\Delta t) = 2r_i(t) - r_i(t+\Delta t) + a_i(t) \Delta t^2$$

Leap-Frog algoritmus

→ jedna diferenciální rovnice 2. řádu jsou dvě dif. rovnice 1. řádu

$$\underbrace{r_i(t+\Delta t)}_A = \underbrace{r_i(t)}_A + \underbrace{v_i(t+\Delta t/2)}_B \Delta t$$

$$\underbrace{v_i(t+\Delta t/2)}_B = \underbrace{v_i(t-\Delta t/2)}_B + \underbrace{a_i(t)}_A \Delta t$$

zrychlení a polohy v celých časech

rychlosti v polovičních časech

→ ekvivalentní bezrychlostnímu Verletovi:

$$r_i(t) = r_i(t-\Delta t) + v_i(t-\Delta t/2) \Delta t$$

$$\text{B} \rightarrow \text{A}: r_i(t+\Delta t) = r_i(t) + v_i(t-\Delta t/2) \Delta t + a_i(t) \Delta t^2$$

$$\rightarrow r_i(t+\Delta t) = 2r_i(t) - r_i(t-\Delta t) + a_i(t) \Delta t^2$$

→ nevýhoda: rychlosti, polohy a zrychlení v různých časech

Verletův rychlostní algoritmus

$$r_i(t+\Delta t) = r_i(t) + v_i(t)\Delta t + \frac{1}{2} a_i(t) \Delta t^2$$

$$v_i(t+\Delta t) = v_i(t) + \frac{1}{2} [a_i(t+\Delta t) + a_i(t)] \Delta t$$

→ opět stejně přesný jako bezrychlostní a leap-frog

→ vysoký účtovatelský komfort

Prediktor-Korektor: Geannův integrátor

→ data zaznamenáváme v super vektoru:

$$R_{i,n}(t) = \frac{h^n}{n!} r_i^{(n)}(t)$$

$$R_{i,n}(t+h) = \sum_{m=0}^{\infty} \frac{h^m}{m!} R_{i,n}^{(m)}(t) = \sum_{m=0}^{\infty} \frac{h^{m+n}}{m!n!} r_i^{(n+m)}(t)$$

$$= \sum_{m=0}^{\infty} \frac{(m+n)!}{m!n!} R_{i,n+m}(t) = \sum_{m=0}^{\infty} \underbrace{\binom{n+m}{n}} R_{i,n+m}(t)$$

matice korektorů je tvořena paskalovským trojúhelníkem ve sloupcích

např. pro čtyř-složkový supervektor:

$$R_i(t) = \begin{pmatrix} r_i \\ h\dot{r}_i \\ \frac{1}{2}h^2\ddot{r}_i \\ \frac{1}{6}h^3\dddot{r}_i \end{pmatrix} \leftarrow \text{matice prediktorů} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

→ ale: rozvoj jen konečný + poč. podmínky

$\Rightarrow$ nutná oprava chyb korektorem, aby r

a $\ddot{r}$ byly spojeny skutečnou $F=ma$

$$E_i(t+h) = \frac{h^2}{2} \underbrace{a_i(r_i^P(t+h))}_{\text{společné síly z predikovaných poloh}} - \frac{h^2}{2} \underbrace{a_i^P(t+h)}_{\text{predikované zrychlení}}$$

stojí výpočet $\rightarrow$ společné síly z predikovaných poloh

predikované zrychlení

→ korigované hodnoty:

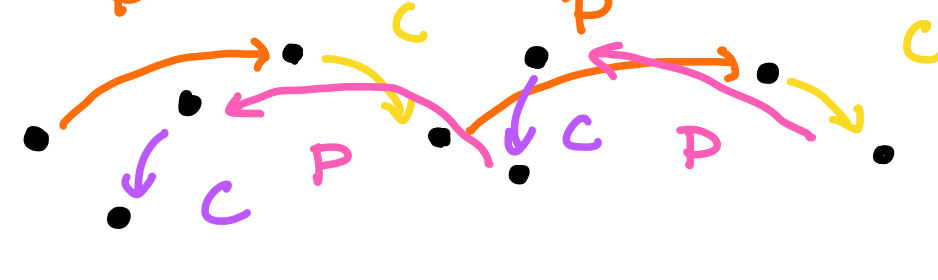
$$R_i^C(t+h) = R_i^P(t+h) + \underline{A} E_i(t+h)$$

A jsou konstanty korektoru, které záleží na řádu

pro 4d: $A = \begin{pmatrix} 1/6 & 5/6 & 1 & 1/3 \end{pmatrix}$

→ není reversibilní,

nezachová energii



Chyba integrátoru a volba časového kroku

krok	chyba 1 kroku	chyba za skutečný čas
Metoda 4. řádu		
h	h^4	h^4
$h/2$	$h^4/16$	$2 \cdot h^4/16 = h^4/8$
$2h$	$16h^4$	$1/2 \cdot 16h^4 = 8h^4$
Metoda 6. řádu		
h	h^6	h^6
$h/2$	$h^6/64$	$2 \cdot h^6/64 = h^6/32$
$2h$	$64h^6$	$1/2 \cdot 64h^6 = 32h^6$

→ integrátory vyššího řádu $\equiv$ ideální na přesné simulace s malým časovým krokem

→ integrátory nižšího řádu $\equiv$ vhodné pro simulace s velkým časovým krokem

→ prediktor-korektor vysokého řádu potlačuje velký stonový vektor

$\Rightarrow$ pro velký počet částic velká paměťová náročnost,

vhodnější spíše Verlet